Semester Project Plan: Faster Shortest Paths on a Changing Graph Using Floyd-Warshall on CPU and GPU

4 weeks | Parallel Floyd-Warshall (OpenMP, CUDA) with incremental updates

What we are doing

Floyd-Warshall finds the shortest path between every pair of nodes in a graph. It takes time proportional to V cubed, so it is slow on large graphs. If only one edge is added or one weight gets smaller, running everything again is a waste. We will test how much faster it is to update only the affected part, on CPU and on GPU, and after how many updates a full recompute becomes the better choice.

What we will claim

We are not claiming a new algorithm. The formula for updating the distance matrix after an edge is added is already known. Our work is to run it with OpenMP and CUDA, measure it, and show for which graph sizes and how many updates it beats a full recompute. We should say this clearly in the report and in the viva.

Scope

We have very little time, so we will do only the items below. Everything else goes under future work.

Item	Plan
Baseline	Serial CPU Floyd-Warshall (all other results are checked against it) and an OpenMP version
GPU	CUDA Floyd-Warshall, simple version first. Tiled version only if time is left
Update	Edge added or weight decreased. Formula: d[i][j] = min(d[i][j], d[i][u] + w + d[v][j]). First on CPU, then on GPU. Applying k updates one after another is our batch update
Edge delete / weight increase	Full recompute only. We will list it as future work in the report
MPI	Not doing it. Future work
Experiments	V = 512, 1024, 2048 (4096 if Colab can handle it). Updates k = 1, 10, 100, 1000, 2000. Time for incremental and for full recompute, on both CPU and GPU
Output	Code on GitHub, a results table and 2 to 3 graphs, a 4 to 6 page report, a short presentation

Week by week plan

We assume little time is available each week. The code is frozen at the end of week 3. Week 4 is only for the report and practice.

Week	Work	Done when
1	On day one, check that a GPU works on Colab or Kaggle. Graph generator, serial Floyd-Warshall, OpenMP version. Also read one CUDA Floyd-Warshall example until it makes sense	Serial and OpenMP give exactly the same result
2	CUDA Floyd-Warshall (simple). Incremental update function on CPU. A script that records timings in a CSV file	CUDA matches serial, first on V = 64 and 128, then on 1024
3	GPU incremental kernel. After every update, check it against a full recompute. Run all experiments on Colab and save the CSV. Freeze the code at the end of the week	All experiment data is in the CSV
4	Graphs, find the crossover point, write the report and slides, practise once or twice	Report and slides are ready

If we fall behind

Cut things in this order: tiled CUDA first, then V = 4096, then the OpenMP thread graphs. These must never be cut: checking against serial, CUDA Floyd-Warshall, the GPU incremental update, and the experiments table.

Things to remember

- Keep all weights non-negative, otherwise the update formula is not correct.
- A Colab session can close in the middle of a run. Keep the code on GitHub and save results to a CSV after every run. Kaggle GPU is the backup.
- Correct first, fast later. Compare every version with the serial one, otherwise a wrong answer can look fast and fine.
- Run every timing 3 to 5 times and take the average. Ignore the first GPU run, it is a warm-up.